

Building Scribe: reading handwritten forms without trusting every answer

I'm building **Scribe** to turn handwritten clinical forms — vitals charts, prescriptions, lab requests — into structured data, entirely on one machine. It's meant for clinics where the records are paper, the connectivity is unreliable, and the data is not allowed to leave the building. So everything runs locally, and the pipeline literally blocks its own network access while it works.

The awkward part hasn't just been reading the handwriting. It's deciding **when to trust the result**. A wrong value in a clinical record looks exactly like a right one, so anything the system isn't sure about has to go to a human instead of into the record. Every answer keeps a link back to the exact spot on the page it came from, so the reviewer sees the handwriting next to the value.

My first approach was the obvious one: ask the model how sure it was.

The median confidence score was **0.95 when it was right — and 0.95 when it was wrong**.

So I needed another way to decide when a person should check a field. This post is the log of finding one: what I tried, where it broke, and what I changed. The failed experiments are kept in, because they taught me the most.

One caveat before any numbers: my test pages come from a [public dataset](#) of Nigerian clinical records (plus a few synthetic pages I built with deliberate traps). Public data may overlap what these models were trained on, so every number here compares configurations against each other — none of it is a claim about real-world accuracy. That claim will come later, from a private held-out set. Full methodology lives in [the repo](#).

A failure you can hold in your hand

Here's my favorite trap from the test set. A lab request form with a specimen row — Blood, Urine, Stool, CSF — and **nothing ticked**:

Specimen type:	☐ Blood	☐ Urine	☐ Stool	☐ CSF
----------------	--------------------------------	--------------------------------	--------------------------------	------------------------------

The right answer is “nothing selected.” Qwen3-VL said “blood.” Its smaller sibling said “blood.” GLM-4.5V said “blood” on the real forms. Gemma 3 read the empty box glyph as a tick mark. Four model families, all quietly convinced somebody must have wanted a blood test — and all of them confident about it.

That's the shape of the whole problem. The dangerous failure isn't the model saying “I can't read this.” It's the model inventing something plausible and rating it 0.95.

What I tried, in order

Cropping each field before reading it

The problem I was chasing: the model kept “borrowing” answers from elsewhere on the page — most often grabbing a date out of the observations table when the form had no date at all.

What I changed: instead of showing the model the whole page for every field, one cheap call first locates every field on the page, and then each field is read from a small cropped image of just that region.

Did it help? Dramatically, where I expected it to: invented dates fell from 67% to 17%, tables read twice as well, and everything got 2.5× faster. But it *hurt* fields whose meaning lives in the printed label next to them — hospital numbers and names got worse, because if the crop is slightly off, the answer is simply gone.

So I stopped choosing and **routed by field type:** tables, dates and checkboxes read from crops; names, numbers and free text read from the full page. That beat both pure approaches and became the default.

Reading everything twice

The problem: confidence couldn’t tell me which answers to distrust — but I noticed that when two readings of the same field *disagreed*, one of them was almost always wrong.

What I changed: every field the pipeline was about to accept without review gets read a second time, from the other source (crop if the first read used the page, and vice versa). Disagreement sends the field to the human queue.

Did it help? More than anything else I tried. The accuracy of answers allowed to skip human review jumped from 79% to 93%, and silent errors — wrong values that sailed through — fell from 21% to 7%. The models never read any better. The *system* got better at knowing when they were wrong.

Trying a different model for the second read

Same-model double-reading has a blind spot: a bias both reads share can never be caught. So I gave the second read to a different, smaller model that runs alongside the main one.

That closed the worst gap I had. Handwritten names had been the scariest failure — *Okafor* misread as *Okaro* with total confidence — and the small model misreads names *differently*, so every one of those now surfaces as a disagreement. Trusted-output accuracy reached 95%.

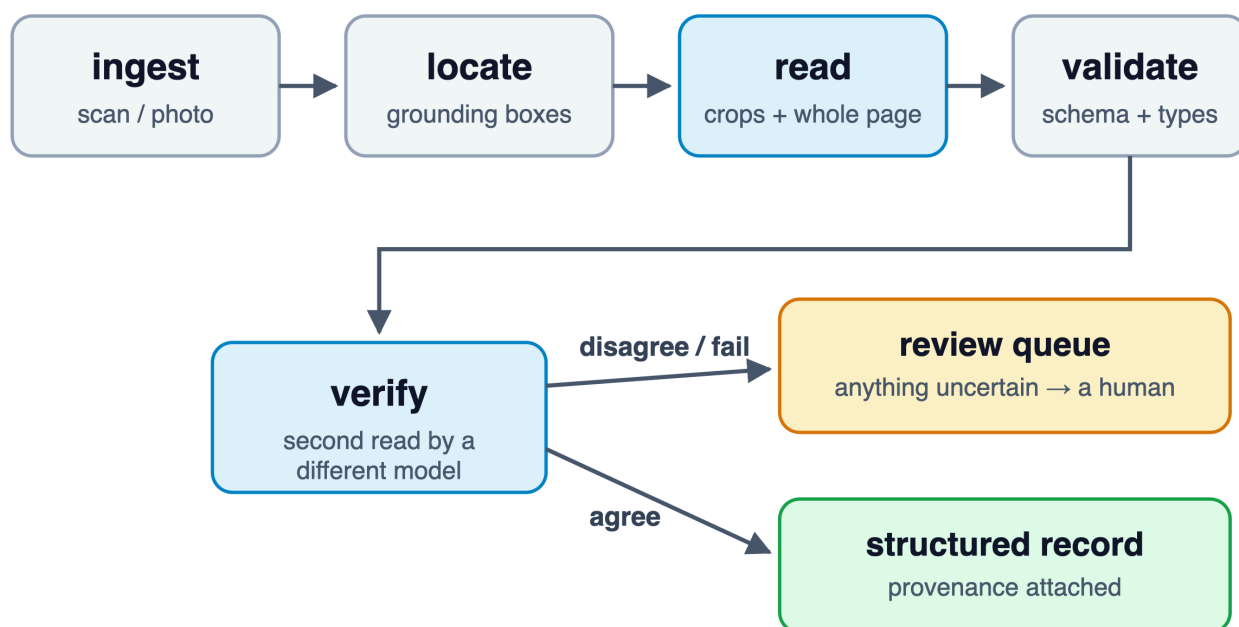
Then I tried fancier second readers — a different model family, a newer generation. Three experiments, three statistical ties. Apparently, once the gate design is sound, almost any competent second reader lands in the same place. I kept the small, fast one and stopped shopping.

Swapping in a newer main model

A new model generation ([Qwen3.8-27B](#)) was the only change in the whole log that made the *reading itself* better: exact accuracy went from 75% to 81%, the borrowed-date problem disappeared entirely, and invented values halved. It's now the default, at the cost of being slower per field.

And one more model taught me something by losing. Meta's Muse Glimmer read this handwriting *worse* than everything else — but it invented **nothing**. Zero hallucinated values, the only model of five families to abstain properly on empty fields. It won't be the main reader, but it's the obvious engine for a future checkbox-detection stage, because the checkbox trap above defeated every other model I tried.

Where that leaves the system



One machine, fully offline — a socket guard blocks every connection off loopback.

Nothing reaches a record unreviewed unless two models independently agree. On the benchmark set, the shipping configuration reads 81% of fields exactly right — but the answers it *lets through without review* are 96% right, and only about 2% of empty fields end up with an invented value slipping past everyone. The gap between those numbers is the review queue doing its job: roughly a quarter of fields go to a human, which is the honest price of the other numbers.

The lesson I keep coming back to: **counting how often the model hallucinates grades the model. Counting how many hallucinations escape grades the system** — and the system is the thing you actually ship.

What still needs work

- **The checkbox problem** is unsolved by model choice — it needs an explicit detection step (likely powered by the one model that refuses to invent).
- **The real test is still ahead:** a held-out set of hand-filled pages on real target forms, created after every model's training cutoff, never published. Those produce the only numbers I'll stand behind — plus a contamination check, because one of the models here was released *after* my public test data was.
- Part 2 of this series already exists: I ambushed the two best configurations with an external dataset nobody tuned for. Reading collapsed; something more interesting held.

The details, for engineers

The full nine-configuration scoreboard (with per-metric definitions), hardware, pinned model revisions, latency tables, and every per-field result live in [docs/bench.md](#) in the repo. The short version:

Configuration	s/field	Exact	Trusted-output acc	Escaped inventions
Whole-page baseline	9.8	75%	75%	20%
Routed crops	6.4	76%	79%	12%
+ second read, same model	9.9	76%	93%	6%
+ second read, different model	10.0	76%	95%	2%
+ newer main model (<i>shipping</i>)	17.3	81%	96%	2%

Benchmark gold set: [sohakanu/scribe-flow-gold](#) · Code: [github.com/kod201/scribe](#)

This is part 1 of a series. Part 2: *One imperfect page per hundred* — the external evaluation on data nobody tuned against.

Every name on every sample page shown or discussed is fictional. All extraction ran locally on one machine; no page left it.